

DST Informatique Luna MP2I

I - Représentation binaire gauche des entiers naturels

1.

Comme la représentation gauche de m est $\sum_{n=0}^{N-1} g_n(2^{n+1} - 1)$,

alors si la représentation gauche de m est 1 suivi de c zéros, alors $m = 2^{c+1} - 1$

La représentation standard d'un $m = 2^n$ est 1 suivi de n zéros, et celle de $2^n - 1$ est un enchaînement de $n - 1$ uns,

Donc la représentation std d'un entier dont une représentation gauche est $10 \dots 0$ avec c zéros est $1 \dots 1$ avec $c - 1$ uns.

Si un entier $m' - 1$ à une représentation gauche qui est $20 \dots 0$ avec c zéros, alors m' a une représentation gauche de $10 \dots 0$ avec $c + 1$ zéros.

La représentation standard de m' est $1 \dots 1$ avec c uns,

Donc celle de $m' - 1$ est $1 \dots 10$ avec $c - 1$ uns.

2.

Le plus grand entier sur N chiffres est $1 \dots 1$ avec N uns en représentation standard,

en représentation gauche il s'agit de $20 \dots 0$ avec $N - 1$ zéros.

Ce nombre ci dessus est donc $M = \sum_{n=0}^{N-1} g_n(2^{n+1} - 1)$

Comme vu ci dessus, $M + 1$ s'écrit $1 \dots 1$ avec $N + 1$ uns en représentation standard, ce qui est $2^{N+1} - 1$

donc $M_N = 2^{N+1} - 2$

5.

```
int rg_to_int (rg g) {  
    int m = 0;  
    for (int i = 0; g.chiffres[i] == true || g.chiffres[i] == false;  
i++) {  
        if (g.chiffres[i] == true) {  
            m += (1 << i+1) - 1;  
        }  
    }  
    if (position > -1){  
        m += (1 << position+1) - 1;  
    }  
    return m;  
}
```

6.

(a)

$\mathbf{g}$ est une représentation gauche valide si celui ci vérifie les 4 conditions énoncées dans le 1.1

condition 1 : $\mathbf{g}$ est une représentation gauche valide car l'unique 2 est changé en 0 qui appartient bien à $\{0, 1, 2\}$,

si il n'y a aucun 2 initialement, alors $g_0 = 2$ et les g_k avec $k \in \llbracket 1, N-1 \rrbracket$ vérifient $g_k \in \{0, 1\}$ car ils sont inchangés

condition 2 :

Cas 1 : Uniquement g_0 change en 2, et $m' = \sum_{n=0}^{N-1} g_n(2^{n+1} - 1)$

Cas 2 : g_p change en 2 et g_{p+1} est incrémenté de 1 et $m' = \sum_{n=0}^{N-1} g_n(2^{n+1} - 1)$

condition 3 :

si il y a initialement un 2 en g_p , alors $g_p = 0$ et g_{p+1} peut valoir 2 et c'est le seul qui peut potentiellement valoir 2, donc la condition est vérifiée,

sinon g_0 peut valoir 2 et c'est le seul qui peut potentiellement l'être, donc la condition est quand même vérifiée

condition 4 :

si il n'y a pas de 2 initialement, alors il peut y avoir un unique mais il est forcément en g_0 , donc il n'existe pas d'indices inférieurs positifs, donc la condition est forcément vérifiée

sinon : Comme $g_p = 2$ initialement et que l'on suppose la condition vérifiée, alors $\forall n \in \llbracket 0, p-1 \rrbracket, g_n$ nul
 Or g_p devient 0 et g_{p+1} est le seul a potentiellement valoir 2, donc la condition est toujours vérifiée avec $p' = p + 1$

Donc **g** est toujours une représentation gauche valide après l'exécution de l'algorithme mystère

(b)

Dans le cas où il n'y a aucun 2 présent initialement, $g_0 (\in \{0, 1\})$ devient $g_0 + 1 (\in \{1, 2\})$ ce qui fait que $m' = m + 1$

Sinon, g_p devient un 0 et on ajoute 1 à la prochaine "dizaine", ce qui revient aussi à $m + 1$

7.

```
bool rg_incr(rg* s) {
    bool b = true;
    if ((s -> position) == -1) {
        if ((s -> chiffres[0]) == true) {
            s -> position = 0;
        }
        else {
            s -> chiffres[0] = true;
        }
    }
    else {
        if (s -> chiffres[(s -> position)+1] == true) {
            s -> position ++;
            s -> chiffres[(s -> position)] = false;
        }
        else {
            if (chiffres[(s -> position)+1] == false) {
                s -> position = -1;
                s -> chiffres[(s -> position)+1] = true;
            }
            else {
                b = false;
            }
        }
    }
    return b;
}
```

8.

Il y a 5 cas différents :

- Il n'y a pas de 2 initialement et `s -> chiffres[0] = true` :
il suffit de modifier `position` afin qu'il vaille 2, complexité constante car `s -> position = 0;` ne dépend pas de N
- Il n'y a pas de 2 initialement et `s -> chiffres[0] = false` :
il suffit de modifier g_0 pour qu'il vaille true, complexité constante car `s -> chiffres[0] = true;` ne dépend pas de N
- il y a un 2 initialement et `s -> chiffres[(s -> position)+1]` vaut true
 g_{p+1} valait initialement 1, donc on le modifie à 2 et on change g_p pour qu'il vaille 0,
complexité constante car `s -> position ++;` et `s -> chiffres[(s -> position)] = false;` ne dépendent pas de N
- il y a un 2 initialement et `s -> chiffres[(s -> position)+1]` vaut false
On change g_{p+1} de 0 à 1 et on change `s -> position` en -1 .
Complexité constante car les deux changements ne dépendent pas de N
- il y a dépassement d'entiers
On change `b` à false, complexité constante

Conclusion

L'incrémentaion a un coût de complexité en temps dans le pire cas constant

la complexité en temps de l'incrémentaion sur la représentation binaire usuelle a un coût linéaire sur N

dans le pire cas car on doit parcourir un certain nombre de chiffres pour la retenue

Si $m = 11 \dots 1$ alors $m + 1 = 10 \dots 0$ avec autant de 0 qu'il y avait de 1 initialement

II - Arbres binaires parfaits

10.

```
int hauteur(arb* a) {  
    if (a == NULL) {  
        return -1;  
    }  
    else {  
        if hauteur(a -> gauche) >= hauteur(a -> droit){  
            return 1+hauteur(a -> gauche);  
        }  
        else {  
            return 1+hauteur(a -> droit);  
        }  
    }  
}
```

C

11.

```
arb* noeud(int v, arb* a_g, arb* a_d) {  
    arb* a = malloc(sizeof(arb));  
    a -> valeur = v;  
    a -> gauche = a_g;  
    a -> droit = a_d;  
    return a;  
}
```

C